

TƯ DUY PHẢN BIỆN VỀ SOFTWARE DESIGN

MSS: 23120262

Tổng Dương Thái Hòa

Phần I. Bóc tách ngộ nhận trong thiết kế phần mềm

Chọn 2 trong 4 nhận định sau và phân tích vì sao nhận định đó đúng một phần nhưng nguy hiểm nếu tin tuyệt đối.

Nhận định

- a) “Càng tách nhiều class càng đúng SRP.”
- b) “Cứ dùng interface là sẽ giảm coupling.”
- c) “Muốn dễ mở rộng thì nên thiết kế sẵn mọi abstraction từ đầu.”
- d) “Code không lặp là code tốt, nên DRY luôn ưu tiên hơn KISS.”

Yêu cầu cho mỗi nhận định

- Chỉ ra phần **đúng**.
- Chỉ ra phần **sai hoặc bị tuyệt đối hóa**.
- Nêu 1 tình huống phản ví dụ.
- Viết lại nhận định đó thành một phiên bản **chính xác hơn** trong 3–4 câu.

Phản biện:

- a) “Càng tách nhiều class càng đúng SRP.”
 - **Phần đúng:** SRP yêu cầu mỗi class chỉ có một lý do để thay đổi. Tách class có thể giúp giảm coupling, tăng testability, nhưng khi một class ôm quá nhiều việc, thay đổi ở một chỗ sẽ kéo theo rủi ro ở nhiều chỗ khác.
 - **Phần nguy hiểm:** SRP không được tách class dựa theo số class nghĩ là sẽ có, mà dựa vào số lý do cần tách. Một module có thể chỉ là 1 class nhưng vẫn cohesive nếu mọi method cùng phục vụ một lý do thay đổi. Ngược lại, tách 10 class siêu nhỏ mà chỉ để

“cho đúng SRP” theo một cách lý thuyết thì lại có thể làm flow code bị chững, và nhiều đến mức như thế thì việc đọc code khá mệt thì phải đọc hiểu từng loại class nữa.

- **Phản ví dụ:** Một class Money có cả add(), subtract(), toCurrency(), 3 method nhưng một lý do thay đổi duy nhất là thay đổi business logic về tiền tệ. Tách ra chỉ làm code phân mảnh vô nghĩa.
- **Viết lại:** Tách class là đúng khi nó làm giảm số lý do thay đổi của từng thành phần. SRP không yêu cầu chia nhỏ em đa, mà yêu cầu gom đúng những gì thay đổi cùng nhau. Nếu tách quá mức làm luồng nghiệp vụ khó đọc hơn, thì đó không còn là thiết kế tốt.

c) “Muốn dễ mở rộng thì nên thiết kế sẵn mọi abstraction từ đầu.”

- **Phản đúng:** OCP chỉ ra module nên mở với mở rộng và đóng với sửa đổi. Để đạt được điều này, ta cần abstraction ở những chỗ muốn sửa đổi code. Nếu không chuẩn bị gì cả, khi yêu cầu mới đến, ta buộc phải chen vào code đang chạy ổn - đầy tính rủi ro về mặt đảm bảo code liệu còn chạy đúng nữa hay không. Về mặt này, "thiết kế sẵn abstraction" là hợp lý.
- **Phản sai:** Vấn đề nằm ở chữ "mọi" và "từ đầu". Thiết kế mọi abstraction từ đầu đồng nghĩa với việc chúng ta đang đoán hệ thống sẽ thay đổi theo hướng nào - mà trong thực tế, dev nổi tiếng tệ ở việc dự đoán yêu cầu tương lai. Từ đó dẫn đến việc abstraction được thiết kế cho kịch bản không bao giờ xảy ra → code phức tạp vô ích; hoặc abstraction sai hướng → khi yêu cầu thật đến, bạn phải vừa xóa abstraction cũ vừa tạo mới, tốn gấp đôi công sức. Hoặc team mới vào dự án phải mày mò tìm hiểu hàng tá interface, abstract class, factory mà không ai dùng, và không ai dám xóa vì code đang chạy làm ơn đừng sửa.
- **Phản ví dụ:** Một team startup làm 1 cái ecommerce. Ngay từ sprint đầu tiên, họ tạo các IShippingStrategy, IShippingCostCalculator, AbstractShippingProvider, ShippingProviderFactory, ShippingProviderRegistry để chuẩn bị cho tương lai sẽ tích hợp thêm nhiều nhà vận chuyển. Mấy tháng sau, họ vẫn chỉ dùng ViettelPost, và cả hệ thống giao hàng phải re-write vì model ban đầu không match với cách API của ViettelPost hoạt động trong thực tế. Toàn bộ abstraction trở thành rác - nhưng team đã bỏ 3 sprint để build và maintain nó.
-
- **Viết lại:** Mở rộng tốt không đến từ việc chuẩn bị sẵn mọi abstraction, mà đến từ việc giữ code đủ sạch để dễ refactor khi yêu cầu thật xuất hiện. Abstraction nên được tạo tại thời điểm bạn có đủ bằng chứng, không phải dựa trên việc dự đoán.

Phần II. Chẩn đoán một thiết kế có vấn đề

Tình huống

Một nhóm sinh viên viết hệ thống quản lý đơn hàng với class sau:

```
class OrderManager {  
    public void createOrder(Order order) { ... }  
}
```

```
public void validateOrder(Order order) { ... }  
public void saveToDatabase(Order order) { ... }  
public void sendConfirmationEmail(Order order) { ... }  
public void printInvoice(Order order) { ... }  
public double calculateShippingFee(Order order) { ... }  
public void exportExcelReport() { ... }  
}
```

Câu hỏi

Hãy đánh giá class trên dưới góc nhìn của **cohesion, coupling, SRP, testability, maintainability**.

Yêu cầu bắt buộc

1. Phân tích vì sao class này có thể bị xem là **low cohesion**.
2. Chỉ ra ít nhất 4 lý do thay đổi khác nhau của class này.
3. Chọn 2 vấn đề nghiêm trọng nhất và giải thích vì sao chúng nguy hiểm hơn các vấn đề còn lại.
4. Đề xuất refactor ở mức thiết kế:
 - tách thành những thành phần nào,
 - thành phần nào nên giữ ở domain,
 - thành phần nào là infrastructure/service.
5. Nêu 1 phản biện ngược với chính phương án refactor của anh/chị, ví dụ: “tách quá mức sẽ gây phân mảnh code”.

Phản biện

1. Vì sao class này low cohesion?

Cohesion đo mức độ các thành phần bên trong một module cùng hướng về một mục tiêu, cùng thao tác trên cùng nhóm dữ liệu. OrderManager vi phạm điều này ở mức cơ bản:

- validateOrder và calculateShippingFee thao tác trên business rule của đơn hàng, cái này là domain logic rồi.
- saveToDatabase thao tác trên persistence layer, cần phải biết connection string, ORM config, transaction management.
- sendConfirmationEmail cần biết SMTP server, email template, encoding.
- printInvoice phụ thuộc vào rendering engine, font, layout hóa đơn.

- `exportExcelReport` phụ thuộc vào thư viện Excel, format cột, header.

Ngoài ra, các method này không chia sẻ state nội bộ (internal state) có ý nghĩa. Ví dụ như `sendConfirmationEmail` không cần biết cách `calculateShippingFee` hoạt động.

`exportExcelReport` thậm chí không nhận `Order` làm tham số. Dễ hiểu hơn thì nếu bỏ bất kỳ method nào ra, các method còn lại sẽ không bị ảnh hưởng, vì thế cái này là dấu hiệu rõ nhất của low cohesion.

2. Ít nhất 4 lý do thay đổi khác nhau

	Lý do thay đổi	Method bị ảnh hưởng	Actor yêu cầu
1	Quy tắc validate đơn hàng thay đổi (thêm rule kiểm tra tồn kho, giới hạn số lượng...)	<code>validateOrder</code>	Business Analyst / Product Owner
2	Chính sách phí ship thay đổi (free ship trên 500k, phí theo vùng miền...)	<code>calculateShippingFee</code>	Team vận hành / logistics
3	Đổi database bắt chọt (MySQL → PostgreSQL)	<code>saveToDatabase</code>	Team infrastructure / DBA
4	Đổi email provider hoặc template (SendGrid → SES, thêm tiếng Anh...)	<code>sendConfirmationEmail</code>	Team marketing / CX
5	Thay đổi format hóa đơn (thêm QR code, đổi layout theo quy định thuế)	<code>printInvoice</code>	Kế toán / pháp lý
6	Thêm cột, thay đổi format Excel report	<code>exportExcelReport</code>	Team quản lý / ban giám đốc

3. Hai vấn đề nghiêm trọng nhất

Trộn domain logic với infrastructure concern.

`validateOrder`, `calculateShippingFee` là business rule, chúng định nghĩa đơn hàng hợp lệ trông như thế nào, phí ship tính ra sao. Tụi nó nên thuần túy là vậy thôi, không phụ thuộc gì ngoài domain model. Nhưng khi chúng nằm cùng class với `saveToDatabase` và `sendConfirmationEmail`, mọi thay đổi infrastructure đều có nguy cơ chạm vào business logic theo kiểu sửa lộn, ví dụ ai đó thêm try-catch cho email, vô tình nuốt exception của validation. Hoặc import thư viện email trong cùng class khiến build time tăng, dependency tree phình ra, và domain class bị ô nhiễm bởi third-party library.

Đây là vi phạm nguyên lý dependency direction: domain layer, tầng giá trị nhất và ổn định nhất lại phụ thuộc vào infrastructure, tầng dễ thay đổi nhất.

Testability gần như bằng không.

Không thể test `calculateShippingFee` một cách bình thường được vì class constructor có thể yêu cầu database connection, SMTP config, printer driver. Chúng ta lại phải mock tất cả những thứ không liên quan chỉ để test một phép tính shipping fee. Hệ quả là phải:

- Test chạy chậm (phải setup/teardown database, email mock).
- Test brittle (thay đổi email provider → test shipping fee fail vì mock cũ không khớp).
- Ông dev nào đó (như em) lười viết test → bug lọt → vòng xoáy chất lượng đi xuống.

4. Đề xuất refactor

Domain layer (thuần business logic, không phụ thuộc framework/library ngoài):

- Order - entity, chứa data và behavior gắn liền (vd: `order.totalAmount()`)
- OrderValidator - kiểm tra rule nghiệp vụ (đơn hàng hợp lệ?)
- ShippingFeeCalculator - tính phí ship theo policy (thuần tính toán).

Infrastructure layer:

- OrderRepository - persistence, chịu trách nhiệm save/load từ database.
- EmailNotificationService - gửi email xác nhận.
- InvoicePrintService - render và in hóa đơn.
- ExcelReportExporter - export report dạng Excel.

Application layer (điều phối luồng nghiệp vụ):

- PlaceOrderUseCase (hoặc OrderApplicationService) - orchestrator:
 1. Gọi OrderValidator.validate(order)
 2. Gọi ShippingFeeCalculator.calculate(order)
 3. Gọi OrderRepository.save(order)
 4. Gọi EmailNotificationService.sendConfirmation(order)

Tại sao lại nên làm như vậy, đơn giản là domain layer sẽ sẽ không phải import bất kỳ thư viện nào khác ngoài ngôn ngữ core → dễ test, ổn định, thay đổi ít. Infrastructure layer encapsulate technology choice → đổi DB/email chỉ sửa ở chỗ này. Application layer chỉ là "gọi đúng thứ tự" → dễ làm, dễ hiểu.

5. Phản biện ngược

Refactor ở trên tạo ra 7-8 class + 1 orchestrator, từ ban đầu chỉ có 1 class. Với team 2-3 người, dự án nhỏ, requirement chưa ổn định, overhead này có tác hại là:

- Navigate giữa 8 file tốn thời gian hơn scroll 1 file.
- Luồng xử lý bị "nhảy" qua nhiều class → developer mới mất thời gian trace flow.
- Nếu requirement thay đổi lớn (pivot từ order management sang subscription model), toàn bộ cấu trúc vừa tách có thể phải tách lại hoặc gộp lại theo cách khác.
- DI container setup cho 8 thành phần chỉ để chạy một flow `createOrder` → giống như lấy dao mổ trâu gọt dưa hấu vậy á, phức tạp hóa vấn đề.

→ Theo mình, refactor là đúng hướng, nhưng mức độ tách nên phụ thuộc vào quy mô team và độ trưởng thành của requirement. Với team nhỏ, có thể bắt đầu bằng 3-4 class (tách domain vs infrastructure) thay vì full 8 class ngay lập tức, rồi tách tiếp khi cần.

Phần III. Không phải cứ “đúng SOLID” là tốt hơn

Tình huống

Một giảng viên phản biện hệ thống thanh toán của sinh viên:

```
class PaymentProcessor {  
    public void processPayment(String type, double amount) {  
        if (type.equals("CREDIT_CARD")) { ... }  
        else if (type.equals("PAYPAL")) { ... }  
        else if (type.equals("MOMO")) { ... }  
    }  
}
```

Một sinh viên đề xuất sửa ngay thành:

- `PaymentMethod` interface,
- `CreditCardPayment`, `PayPalPayment`, `MomoPayment`,
- thêm `PaymentFactory`,
- thêm `PaymentRegistry`,
- thêm `PaymentStrategyResolver`,
- thêm `AbstractPaymentHandler`,
- thêm `BasePaymentAuditDecorator`.

Câu hỏi

Anh/chị hãy đánh giá xem đề xuất này là:

- refactor hợp lý,
- over-engineering,
- hay đúng ý nhưng làm quá tay.

Yêu cầu bắt buộc

Bài làm phải trả lời đủ các ý sau:

1. Đoạn code ban đầu vi phạm nguyên lý nào? Giải thích rõ.
2. Trong đề xuất mới, thành phần nào là **thực sự cần thiết**, thành phần nào có dấu hiệu **YAGNI**.
3. Phân tích xung đột giữa:
 - **OCP/DIP**
và
 - **KISS/YAGNI**
4. Nếu hệ thống hiện tại chỉ có đúng **2 phương thức thanh toán** và 6 tháng qua không đổi, anh/chị có refactor ngay không? Vì sao?
5. Nếu công ty chuẩn bị tích hợp thêm 8 cổng thanh toán mới trong 3 tháng em, câu trả lời của anh/chị có thay đổi không?
6. Kết luận cuối cùng của anh/chị phải nêu:
 - **phiên bản thiết kế nào nên áp dụng ngay bây giờ,**
 - **phiên bản nào để dành cho giai đoạn sau.**

Phản biện

1. Vi phạm

Vi phạm OCP: Mỗi lần thêm phương thức thanh toán mới, dev phải mở processPayment ra, thêm một nhánh else if. Class không "đóng với sửa đổi" - mà đúng ra, thêm loại thanh toán mới (extension) không nên buộc sửa code cũ (modification).

Vi phạm SRP ở mức nhẹ: Class biết chi tiết xử lý của CREDIT_CARD, PAYPAL, MOMO - ba actor khác nhau (đội tích hợp từng cổng) đều có thể gây thay đổi trong cùng một method.

2. Thành phần nào thực sự cần, thành phần nào YAGNI?

Thành phần đề xuất	Đánh giá	Lý do
PaymentMethod interface	Cần thiết	Đây là abstraction cốt lõi, giải quyết OCP. Mỗi cổng implement interface này.

Thành phần đề xuất	Đánh giá	Lý do
CreditCardPayment, PayPalPayment, MomoPayment	Cần thiết	Concrete implementation, thay thế if/else.
PaymentFactory	Có thể cần, nhưng chưa chắc	Nếu dùng DI container, factory là thừa - container đã resolve dependency. Nếu không dùng DI, một Map<String, PaymentMethod> đơn giản đủ rồi. Factory pattern chỉ cần khi logic khởi tạo phức tạp (cần config, credential khác nhau cho mỗi cổng).
PaymentRegistry	YAGNI	Registry ám chỉ plugin architecture - dynamic registration, runtime discovery. Với 3 cổng cố định, hardcode wiring đủ rồi.
PaymentStrategyResolver	YAGNI	Đây chỉ là đổi tên cho Map.get(type) hoặc switch statement - thêm một layer indirection mà không thêm value.
AbstractPaymentHandler	YAGNI	Abstract base class chỉ hợp lý khi có shared logic đáng kể giữa các implementation. Chưa có bằng chứng 3 cổng này share gì - mỗi cổng gọi API khác nhau hoàn toàn.
BasePaymentAuditDecorator	YAGNI rõ nhất	Audit chưa phải requirement. Decorator pattern chỉ đáng khi cross-cutting concern (logging, audit, retry) tồn tại thực sự và cần apply uniform cho nhiều class.

3. Phân tích xung đột OCP/DIP vs KISS/YAGNI

OCP/DIP: "Hãy depend on abstraction, đóng với modification, mở với extension."

KISS/YAGNI: "Đừng thêm complexity cho yêu cầu chưa tồn tại."

Xung đột nằm ở mức độ đầu tư trước (upfront investment):

- OCP/DIP khuyến khích đầu tư sớm vào abstraction → chi phí trả trước, lợi ích gặt sau.
- KISS/YAGNI khuyến khích trì hoãn đầu tư → giữ code đơn giản nhất có thể cho yêu cầu hiện tại.

Cách giải quyết:

- Nếu xác suất thay đổi cao + chi phí refactor sau lớn → đầu tư abstraction sớm (OCP thắng).
- Nếu xác suất thay đổi thấp + chi phí thêm abstraction bây giờ lớn → giữ đơn giản (KISS thắng).
- Nếu xác suất thay đổi cao + chi phí refactor sau nhỏ (code clean, có test) → vẫn giữ đơn giản, refactor khi cần (YAGNI thắng, vì code tốt cho phép refactor dễ).

4. Nếu chỉ có 2 phương thức thanh toán, 6 tháng không đổi

Không refactor lớn ngay vì:

- 2 phương thức + 6 tháng ổn định = bằng chứng thực nghiệm rằng requirement không biến động ở chỗ này. Xác suất cần extend thấp.
- If/else với 2 nhánh hoàn toàn chấp nhận được về readability. Một người đọc code mất 5 giây hiểu flow - nhanh hơn nhiều so với trace qua interface → factory → concrete class.
- Chi phí refactor hiện tại (tạo interface, tách class, wire DI, update test) > lợi ích nhận được (linh hoạt cho yêu cầu chưa có).

5. Nếu sắp tích hợp thêm 8 cổng trong 3 tháng t

Lúc này sẽ refactor vì:

- If/else với 11 nhánh (3 cũ + 8 mới) là nightmare - code dài, khó đọc, dễ conflict khi 2 dev cùng sửa.
- Mỗi cổng thanh toán có API, credential, error handling riêng → mỗi nhánh if/else sẽ phình ra 50-100 dòng → method processPayment có thể lên 500+ dòng.
- Test coverage giảm vì mọi thứ nằm trong 1 method lớn → mock khó, assertion khó isolate.

6. Kết luận phiên bản thiết kế

PaymentMethod interface + concrete class per gateway + simple map/DI wiring. Đây là điểm cân bằng thực tế: giải quyết OCP, giữ KISS, không vi phạm YAGNI. Tổng cộng ~5 file, dev mới hiểu trong 10 phút. Để dành cho giai đoạn sau gồm Factory (khi creation logic complex), Registry (khi cần plugin), Abstract base (khi có shared pattern), Decorator (khi có cross-cutting concern). Mọi thứ chỉ thêm khi có bằng chứng thực tế, không phải khi có dự đoán.

Phần IV. Phản biện các “đề xuất cải tiến nghe có vẻ đúng”

Đọc 3 đề xuất sau và chọn **2 đề xuất** để phản biện.

Đề xuất 1

“Để đúng DIP, mọi class trong hệ thống đều nên phụ thuộc vào interface, kể cả các class rất nhỏ chỉ có một implementation.”

Đề xuất 2

“Để đúng ISP, interface càng nhỏ càng tốt, mỗi method nên tách thành một interface riêng.”

Đề xuất 3

“Để tránh vi phạm LSP, tốt nhất là hạn chế kế thừa, hầu như chỉ nên dùng composition.”

Yêu cầu cho mỗi đề xuất

- Nêu vì sao đề xuất đó **nghe có vẻ hợp lý**.
- Nêu rủi ro nếu áp dụng cục đoan.
- Cho **1 ví dụ thực tế** khi đề xuất đó là đúng.
- Cho **1 ví dụ thực tế** khi đề xuất đó trở thành sai hoặc lãng phí.
- Kết luận bằng một nguyên tắc cân bằng ngắn gọn của riêng anh/chị.

Đề xuất 1: "Để đúng DIP, mọi class đều nên phụ thuộc vào interface, kể cả class nhỏ chỉ có một implementation."

Vì sao nghe hợp lý:

DIP nói high-level module không nên phụ thuộc low-level module, cả hai nên phụ thuộc abstraction. Slide minh họa Customer chuyển từ phụ thuộc FileLogger sang ILogger - sạch, đẹp, testable. Logic mở rộng tự nhiên là: "vậy mọi thứ đều nên qua interface!"

Rủi ro nếu áp dụng cục đoan:

- Explosion of interfaces: Hệ thống 200 class → 200 interface + 200 implementation. Mỗi interface chỉ có đúng 1 class implement → interface không phải abstraction, nó chỉ là duplicate của class signature trên một file khác.
- Navigation hell: IDE phải "Go to Implementation" thay vì "Go to Definition" → thêm 1 click mọi nơi, mọi lúc. Dev dành 30% thời gian nhảy qua nhảy lại giữa interface và implementation.
- False sense of decoupling: Interface chỉ decouple khi có nhiều hơn 1 implementation thực tế hoặc khi bạn cần test seam. Interface cho DateFormatter khi app chỉ dùng ISO-8601 và sẽ không bao giờ đổi → decoupling trên giấy, coupling y nguyên trong thực tế.
- DI container bloat: Register 200 interface → 200 binding → startup chậm, config dài, bug registration khó debug.

Ví dụ thực tế khi đúng:

IPaymentGateway - có Stripe implementation cho production, có mock implementation cho test, và sắp thêm PayPal. Interface ở đây thực sự tạo seam hữu ích: test nhanh hơn (mock gateway thay vì gọi API thật), deploy linh hoạt hơn (swap gateway theo environment).

Ví dụ thực tế khi sai/lãng phí:

StringUtilHelper interface cho class StringHelper chỉ có method capitalize(). Class này thuần utility, deterministic, không I/O, không side effect. Test nó bằng cách gọi thẳng là đủ - không cần mock, không cần swap, không cần interface. Tạo interface ở đây chỉ tốn 1 file thừa và 1 binding registration thừa.

Một ví dụ mạnh hơn: ITaxCalculator trong ứng dụng nội bộ chỉ phục vụ thị trường Việt Nam, thuế VAT cố định 10%. Tạo interface "phòng khi đổi sang thị trường khác" khi công ty không có kế hoạch ra nước ngoài → YAGNI thuần túy.

Nguyên tắc cân bằng:

Interface xứng đáng tồn tại khi đứng ở volatility boundary - biên giữa ổn định và biến động: I/O (database, file, network), external service (payment, email, SMS), hoặc policy (business rule có khả năng thay đổi cao). Với logic nội bộ, ổn định, chỉ có 1 implementation, concrete class là đủ - và đó không phải vi phạm DIP, mà là áp dụng DIP có chọn lọc, đúng chỗ.

Đề xuất 3: "Để tránh vi phạm LSP, tốt nhất là hạn chế kế thừa, hầu như chỉ nên dùng composition."

Vì sao nghe hợp lý:

LSP (Liskov Substitution Principle) nói rằng subtype phải thay thế được base type mà không làm hỏng behavior. Trong thực tế, rất nhiều kế thừa vi phạm LSP: Square extends Rectangle (classic), ReadOnlyList extends List (throw UnsupportedOperationException ở method add). Gang of Four cũng nói "Favor composition over inheritance." Vậy kết luận tự nhiên: "hạn chế kế thừa = tránh LSP violation."

Rủi ro nếu áp dụng cực đoan:

- Không phải mọi kế thừa đều vi phạm LSP. Kế thừa đúng cách - khi subtype thật sự là-một (is-a) base type về mặt behavior - vẫn là công cụ mạnh nhất để tái sử dụng behavior + đảm bảo substitutability. Cấm kế thừa hoàn toàn = tự bỏ đi một công cụ thiết kế hợp lệ.
- Composition overhead: Composition yêu cầu bạn viết delegation code thủ công. Nếu base class có 15 method và subclass chỉ cần override 1, kế thừa cho bạn 14 method miễn phí. Composition buộc bạn phải wrap cả 15 → boilerplate khổng lồ.

- Semantic loss: Kế thừa biểu đạt rõ ràng mối quan hệ "A là một loại B" - reader hiểu ngay. Composition che giấu mối quan hệ này: "A có một B bên trong" nghe khác hẳn và có thể gây hiểu nhầm về intent.
- Framework/library constraints: Nhiều framework (Java Servlet, Android Activity, React Component trước hooks) buộc bạn phải kế thừa. "Chỉ dùng composition" trong context này là bất khả thi.

Ví dụ thực tế khi đề xuất đúng:

`Stack` extends `Vector` trong Java standard library - đây là kế thừa sai vì `Stack` không phải `Vector` (`Stack` chỉ cho phép `push/pop`, nhưng kế thừa `Vector` lộ ra `add(index)`, `remove(index)` → client có thể insert vào giữa stack → phá vỡ invariant LIFO). Nếu dùng composition (`Stack` chứa `Vector` bên trong, chỉ expose `push/pop/peek`), LSP không bị vi phạm vì `Stack` không claim mình là `Vector`.

Ví dụ thực tế khi đề xuất sai/lãng phí:

`ArrayList` extends `AbstractList` trong Java - đây là kế thừa hoàn toàn hợp lệ. `ArrayList` thỏa mãn mọi contract của `AbstractList`. Template Method pattern ở đây cho phép `AbstractList` define skeleton (`iterator`, `indexOf...`) và subclass chỉ cần implement `get(index)` và `size()`. Nếu thay thế bằng composition, bạn phải copy hàng trăm dòng code utility từ `AbstractList` vào `ArrayList` - duplication thuần túy, không thêm value.

Một ví dụ gần gũi hơn: `PremiumCustomer` extends `Customer` khi `PremiumCustomer` thỏa mọi behavior của `Customer` và chỉ thêm (không sửa) behavior mới (discount calculation). Kế thừa ở đây rõ ràng, đúng LSP, dễ hiểu.

Nguyên tắc cân bằng:

Vấn đề không phải "kế thừa hay composition" - mà là "mối quan hệ giữa hai class là gì?":

- Nếu B thực sự là một loại A (thỏa mọi contract, không cần ẩn method nào) → kế thừa hợp lệ, LSP an toàn.
- Nếu B dùng khả năng của A nhưng không phải A (cần ẩn bớt method, thay đổi precondition/postcondition) → composition đúng.
- Nếu không chắc → bắt đầu với composition (an toàn hơn), chuyển sang kế thừa khi thấy rõ mối quan hệ is-a.

Hạn chế kế thừa là guideline tốt cho người chưa có kinh nghiệm (vì sai kế thừa phổ biến hơn sai composition). Nhưng biến guideline thành luật tuyệt đối là đánh mất công cụ thiết kế - và đó cũng là một dạng over-engineering: tốn effort viết delegation code chỉ vì sợ kế thừa.